

TypeScript Interview Answers

Technical Questions

1. **How has TypeScript improved your development workflow?** TypeScript has improved my workflow by:

- Catching type errors at compile time rather than runtime
- Providing better IDE support with accurate autocomplete
- Making refactoring safer and more efficient
- Improving code documentation through type declarations
- Enabling faster onboarding for new team members
- Enhancing code quality through explicit interfaces
- Enabling better tooling for large-scale applications
- Making code more maintainable and self-documenting

2. **Explain the difference between interfaces and types in TypeScript. Interfaces:**

- Can be extended with `extends` keyword
- Can be merged with multiple declarations (declaration merging)
- Better for defining object shapes and API contracts
- More aligned with OOP principles
- Cannot represent primitives directly

Types:

- Can use union, intersection, and mapped types more flexibly
- Cannot be re-opened for extension after definition
- Can represent primitives, unions, and tuples directly
- More aligned with functional programming
- Support for mapped and conditional types

```
// Interface
interface User {
  name: string;
  age: number;
}

interface User { // Merged with above
  email: string;
}

// Type
type User = {
  name: string;
  age: number;
  email: string;
}
```

3. How do you handle union and intersection types? Union Types (|):

- Represent values that could be one of several types
- Useful for function parameters accepting multiple types
- TypeScript only allows operations that are valid for all types in the union
- Need type guards to access type-specific properties

```
function formatValue(value: string | number) {
  if (typeof value === 'string') {
    return value.trim();
  }
  return value.toFixed(2);
}
```

Intersection Types (&):

- Combine multiple types into one
- Useful for mixins and composition
- Result has all properties from all constituent types
- May lead to never type if incompatible properties exist

```
type Employee = {  
  id: number;  
  name: string;  
};  
  
type Manager = {  
  employees: Employee[];  
};  
  
type ManagerEmployee = Employee & Manager;
```

4. **Explain generics in TypeScript with examples.** Generics allow creating reusable components that work with a variety of types:

```
// Generic function
function identity<T>(arg: T): T {
  return arg;
}

// Generic interface
interface Repository<T> {
  getById(id: string): Promise<T>;
  getAll(): Promise<T[]>;
  create(entity: T): Promise<T>;
}

// Generic class
class Queue<T> {
  private items: T[] = [];

  enqueue(item: T): void {
    this.items.push(item);
  }

  dequeue(): T | undefined {
    return this.items.shift();
  }
}

// Generic constraints
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
```

5. **What are type guards and how do they improve type safety?** Type guards are expressions that perform runtime checks to guarantee type information:

```
// Built-in type guards
function process(value: string | number) {
  if (typeof value === 'string') {
    // TypeScript knows value is string here
    return value.toUpperCase();
  }
  // TypeScript knows value is number here
  return value.toFixed(2);
}

// Custom type guards
interface User {
  name: string;
  email: string;
}

interface Admin {
  name: string;
  permissions: string[];
}

function isAdmin(user: User | Admin): user is Admin {
  return 'permissions' in user;
}

function checkAccess(user: User | Admin) {
  if (isAdmin(user)) {
    // TypeScript knows user is Admin here
    console.log(user.permissions);
  } else {
    // TypeScript knows user is User here
    console.log(user.email);
  }
}
```

They improve type safety by:

- Preventing runtime errors by enforcing proper property access
- Enabling TypeScript to narrow types in conditional branches
- Making code more self-documenting
- Eliminating the need for type assertions

6. How do you approach migrating a JavaScript codebase to TypeScript? A pragmatic approach to migration:

1. Add TypeScript tooling alongside existing JS build process
2. Configure tsconfig.json with allowJs: true and noEmit: false
3. Rename files incrementally (.js to .ts or .tsx)
4. Start with the loosest settings (any for unknown types)
5. Add @ts-ignore comments where needed initially
6. Create interfaces for core data structures
7. Add types to public API functions first
8. Gradually increase strictness in tsconfig.json
9. Use the type-coverage tool to track progress
10. Create a style guide for TypeScript usage

7. **Explain conditional types and utility types. Conditional Types:** Types that depend on a condition, using syntax similar to ternary operators:

typescript

 Copy

```
type NonNullable<T> = T extends null | undefined ? never : T;

type ExtractReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
```

Utility Types: Built-in types that transform other types:

- `Partial<T>`: Makes all properties optional
- `Required<T>`: Makes all properties required
- `Readonly<T>`: Makes all properties readonly
- `Pick<T, K>`: Picks subset of properties
- `Omit<T, K>`: Omits specified properties
- `Record<K, T>`: Creates a type with properties K of type T
- `Exclude<T, U>`: Excludes types in U from T
- `Extract<T, U>`: Extracts types in U from T
- `NonNullable<T>`: Removes null and undefined from T
- `ReturnType<T>`: Extracts return type of function type
- `Parameters<T>`: Extracts parameter types of function type

8. **How do you handle typing for external libraries without TypeScript definitions?**

- Check if `@types/library-name` exists on DefinitelyTyped
- Create your own declaration file (module.d.ts):

```
// module.d.ts
declare module 'untyped-library' {
  export function someFunction(arg: string): number;
  export class SomeClass {
    constructor(options?: any);
    method(arg: string): void;
  }
  export default SomeClass;
}
```

- Use module augmentation to add to existing declarations
- Start minimal with `any` and gradually refine types
- Use typeof with imported values to infer types
- Contribute types back to the community

9. Describe namespace vs. module in TypeScript. Namespaces:

- Internal TypeScript feature for organizing code
- Uses `namespace` keyword
- Compiled to IIFE in JavaScript
- Accessed using dot notation
- Less common in modern TypeScript

```
namespace Validation {
  export interface StringValidator {
    isValid(s: string): boolean;
  }
}
```

Modules:

- Based on ES modules (import/export)
- Each file is its own module
- Better for tree-shaking and bundling
- More aligned with modern JavaScript
- Preferred approach for most applications

```
// validation.ts
export interface StringValidator {
  isValid(s: string): boolean;
}

// app.ts
import { StringValidator } from './validation';
```

10. **What are decorators in TypeScript?** Decorators are special declarations attached to classes, methods, properties, or parameters that can modify their behavior:

```
// Class decorator
function Logger(target: Function) {
  console.log(`Class ${target.name} was created`);
}

// Method decorator
function Log(target: any, propertyKey: string, descriptor: PropertyDescriptor) {
  const original = descriptor.value;
  descriptor.value = function(...args: any[]) {
    console.log(`Calling ${propertyKey} with`, args);
    return original.apply(this, args);
  };
  return descriptor;
}

// Property decorator
function Required(target: any, propertyKey: string) {
  const values = new WeakMap<any, any>();
  Object.defineProperty(target, propertyKey, {
    set(value: any) {
      if (!value) throw new Error(`${propertyKey} is required`);
      values.set(this, value);
    },
    get() {
      return values.get(this);
    }
  });
}

@Logger
class User {
  @Required
  name: string;

  @Log
  greet() {
    return `Hello, ${this.name}`;
  }
}
```

Common uses:

- Dependency injection
- Method interception
- Validation

- Logging and metrics
- Property observers
- Memoization

11. **How do you configure TypeScript for different project requirements?** Key TypeScript configuration options in tsconfig.json:

- **Compilation targets:** target, lib, module
- **Type checking strictness:** strict, noImplicitAny, strictNullChecks
- **Module resolution:** moduleResolution, baseUrl, paths
- **Project structure:** rootDir, outDir, include, exclude
- **Interoperability:** allowJs, declaration, esModuleInterop
- **Source maps:** sourceMap, inlineSourceMap

Different project configurations:

- For libraries: stricter settings, generate declarations
- For applications: might be less strict initially
- For migration projects: incremental strictness
- For Node.js: specific module settings
- For React: JSX support, special lib settings

12. **Explain `unknown` vs. `any` type. any:**

- Opt-out of type checking
- Allows all operations without checks
- Values can flow to other types without assertion
- Properties and methods can be called freely
- Should be avoided when possible

unknown:

- Type-safe counterpart to any
- No operations allowed without type checking
- Requires type assertion or narrowing before use
- Cannot be assigned to other types directly
- Safer for values of uncertain type

```
function processAny(value: any) {
  value.method(); // No error
  return value + 5; // No error
}

function processUnknown(value: unknown) {
  value.method(); // Error: Object is of type 'unknown'

  // Must narrow type first
  if (typeof value === 'object' && value !== null && 'method' in value) {
    value.method(); // OK
  }

  if (typeof value === 'number') {
    return value + 5; // OK
  }
  return String(value);
}
```

Experience-Based Questions

1. **How have you convinced team members to adopt TypeScript?** Converting skeptical team members to TypeScript advocates required a strategic, results-driven approach: **Initial**

Assessment:

- Identified pain points in our JavaScript codebase (runtime errors, refactoring difficulties)
- Gathered metrics on bug types and frequency
- Documented time spent debugging type-related issues

Strategic Implementation:

- Started with a small, isolated feature to demonstrate value
- Created a side-by-side comparison of the same component in JS vs TS
- Gradually integrated TypeScript into the build pipeline without disrupting workflow

Education & Support:

- Ran hands-on workshops tailored to our codebase
- Created a "TypeScript champion" program where interested developers became experts
- Developed a company-specific TypeScript style guide
- Set up pair programming sessions focused on TypeScript conversion
- Established a Slack channel for quick TypeScript questions

Demonstrating Value:

- Built an error catalog showing bugs that TypeScript would have caught

- Showed improved IDE support and developer experience
- Created metrics dashboard tracking:
 - Reduction in runtime type errors
 - Time saved during refactoring
 - Code review efficiency improvements

Addressing Common Objections:

1. "It's too verbose" → Showed how type inference reduces explicit annotations
2. "It slows down development" → Demonstrated long-term velocity improvements
3. "Learning curve is too steep" → Created graduated learning paths
4. "Too academic/theoretical" → Focused on practical benefits to daily work

Success Story: For one particularly skeptical senior developer, I created a focused demo refactoring a problematic area of our codebase that he maintained. When he saw how TypeScript caught three latent bugs during the conversion process and made a complex refactoring significantly safer, he became one of our strongest TypeScript advocates. Within six months, he had personally converted over 30,000 lines of JavaScript to TypeScript. The end result was universal adoption, with our entire frontend team embracing TypeScript and our error rates dropping by 38% in production.

2. **Describe how TypeScript has helped prevent bugs in your applications.** TypeScript has prevented numerous bugs across our applications. Here are specific categories with examples:

Null/Undefined Errors: Before TypeScript, we frequently encountered:

javascript

 Copy

```
function getUsername(user) {  
  return user.profile.name; // Runtime error if profile is undefined  
}
```

With TypeScript and strictNullChecks:

typescript

 Copy

```
function getUsername(user: User): string {  
  // Error caught during compilation: Object is possibly undefined  
  return user.profile.name;  
  
  // Fixed version:  
  return user.profile?.name ?? 'Unknown';  
}
```

API Integration Bugs: When our backend API changed, TypeScript caught mismatches:

```
// Our interface based on API contract
interface User {
  id: number;
  name: string;
  role: 'admin' | 'user' | 'guest';
}

// When backend changed 'role' to 'userRole',
// TypeScript immediately flagged all usage points
```

Refactoring Safety: During a major refactoring of our authentication system:

```
// Before refactoring
function authenticate(credentials: Credentials): Promise<User> {
  // ...
}

// After changing return type
function authenticate(credentials: Credentials): Promise<AuthResult> {
  // ...
}

// TypeScript immediately identified all 37 locations
// that needed updates across our codebase
```

Library Updates: When updating React from v16 to v17, TypeScript identified breaking changes:

```
// This pattern broke in the update
const Component: React.FC<Props> = (props) => {
  // TypeScript flagged children property access changes
  return <div>{props.children}</div>;
};
```

Real-world Bug Prevention: In our e-commerce application, TypeScript prevented a critical bug:

```
// The bug: accidentally using string concatenation with prices
function calculateTotal(items: CartItem[]): number {
  return items.reduce((total, item) => {
    // This would concatenate strings instead of adding numbers
    // if price was accidentally a string
    return total + item.price * item.quantity;
  }, 0);
}
```

TypeScript caught this issue during a refactoring when an API change returned prices as strings instead of numbers, preventing incorrect calculations at checkout. **Measurable Results:**

- 46% reduction in runtime errors after full TypeScript adoption
- 28% fewer QA-reported bugs related to data handling
- 62% decrease in "undefined is not a function" type errors
- Critical payment processing bugs reduced to zero in the last year

These improvements translated to higher customer satisfaction, fewer emergency fixes, and more time for new feature development.

3. How do you balance type safety with development speed? Balancing type safety with development velocity requires pragmatic approaches: **Incremental Type Safety:**

- Start with minimal typing (`allowJs: true`, `noImplicitAny: false`)
- Gradually increase strictness as codebase matures
- Use stricter settings for critical code paths (authentication, payments)
- Implement progressive adoption plan with clear milestones

Strategic Use of `any` and Type Assertions:

- Allow controlled use of `any` in specific scenarios:
 - Prototyping new features
 - Third-party libraries without type definitions
 - Complex legacy code conversion
- Require justification comments with every `any` usage
- Schedule regular "type debt" cleanup sprints

Type Inference Optimization:

- Leverage TypeScript's inference when possible
- Only add explicit types where inference is insufficient
- Use type utilities to reduce repetition:

```
// Instead of duplicating interface properties
function createUser(data: Pick<User, 'name' | 'email'>): User {
  // ...
}
```

Productivity Tools:

- IDE snippets for common type patterns
- Custom ESLint rules to enforce team standards
- Auto-generation tools for repetitive type definitions
- Integration with API schema tools (OpenAPI, GraphQL codegen)

Documentation Balance:

- Focus JSDoc on "why" not "what" (types handle the "what")
- Use TSDoc for complex type relationships
- Create living examples for common patterns

Pragmatic Testing Approach:

- Unit tests focus on behavior, not types
- Types reduce need for certain runtime validation tests
- Integration tests catch issues at boundaries

Team Strategies:

- Type experts review complex typings
- Regular knowledge sharing sessions
- "Type buddies" for pair programming
- Clear documentation for common patterns

Real Application Example: In our customer dashboard, we applied a "concentric rings" approach:

- Core data models and state management: Strictest typing
- UI components: Moderate typing with inference
- Visualization/charting: More flexible typing with interfaces for key points
- Experimental features: Relaxed typing during iteration

This approach let us maintain 92% type coverage while still allowing rapid prototyping where needed. As experimental features stabilized, we incrementally improved their type safety. This balanced approach reduced bugs while maintaining team velocity.